

TDC - Exploring Widgets - L2

Detailed Content - H1 - H4

- [Layouts L2](#)
 - [QGridLayout](#)
 - [QFormLayout](#)
- [Dialogs](#)
 - [QFileDialog](#)
 - [QColorDialog](#)
 - [QFontDialog](#)
- [Widgets L2](#)
 - [QMainWindow](#)
 - [QPixmap](#)
 - [QSpinBox](#)
 - [QScrollArea](#)
 - [QDoubleSpinBox](#)
 - [QSplitter](#)
 - [QDockWidget](#)
- [View Widgets](#)
 - [QListView](#)
 - [QTreeView](#)
 - [QTableView](#)

Detailed Content - H1 - H2

- [Layouts L2](#)
 - [QGridLayout](#)
 - [QFormLayout](#)
- [Dialogs](#)
 - [QFileDialog](#)
 - [QColorDialog](#)
 - [QFontDialog](#)
- [Widgets L2](#)
 - [QMainWindow](#)
 - [QPixmap](#)
 - [QSpinBox](#)
 - [QScrollArea](#)
 - [QDoubleSpinBox](#)
 - [QSplitter](#)
 - [QDockWidget](#)
- [View Widgets](#)
 - [QListView](#)
 - [QTreeView](#)
 - [QTableView](#)

Detailed Content - H1

- [Layouts L2](#)
- [Dialogs](#)
- [Widgets L2](#)
- [View Widgets](#)

Layouts L2

QGridLayout

QGridLayout arranges widgets in a **grid** of rows and columns, much like a table or spreadsheet. Great for forms and tool layouts.

```
1 import sys
2 from PySide2.QtWidgets import QApplication, QWidget, QGridLayout,
  QPushButton
3
4 class Main(QWidget):
5     def __init__(self):
6         super().__init__()
7         self.setWindowTitle("QGridLayout Example")
8
9         layout = QGridLayout()
10        layout.addWidget(QPushButton("Top-Left"), 0, 0)
11        layout.addWidget(QPushButton("Top-Right"), 0, 1)
12        layout.addWidget(QPushButton("Bottom-Left"), 1, 0)
13        layout.addWidget(QPushButton("Bottom-Right"), 1, 1)
14
15        self.setLayout(layout)
16
17 if __name__ == "__main__":
18     app = QApplication(sys.argv)
19     window = Main()
```

```

20     window.resize(250, 150)
21     window.show()
22     app.exec_()

```

💡 Explanation:

- **QGridLayout()** : Creates a grid system to place widgets using **(row, column)** coordinates.
- **addWidget(widget, row, column)** : Places each widget precisely.
- Flexible for both simple and complex UI arrangements.

QFormLayout

- **QFormLayout** arranges widgets in a two-column layout: labels on the left and input fields (or widgets) on the right.
- Great for building clean, readable forms (e.g., publish forms, settings panels).

```

1  import sys
2  from PySide2.QtWidgets import QApplication, QWidget, QFormLayout,
   QLabel, QLineEdit, QPushButton
3
4  class Main(QWidget):
5      def __init__(self):
6          super().__init__()
7          self.setWindowTitle("QFormLayout Example")
8
9          layout = QFormLayout()
10         layout.addRow("Name:", QLineEdit())
11         layout.addRow("Department:", QLineEdit())
12         layout.addRow("Version:", QLineEdit())
13         layout.addRow(QPushButton("Submit"))
14
15         self.setLayout(layout)
16
17 if __name__ == "__main__":
18     app = QApplication(sys.argv)
19     window = Main()
20     window.resize(300, 150)
21     window.show()
22     app.exec_()
23

```

Explanation

- **QFormLayout()**
 - Creates a layout where each row has a label and a field, making it perfect for structured forms.
- **addRow(label, widget)**
 - Adds a label and corresponding input field to the layout, side-by-side.
- **addRow(widget)**
 - Adds a full-width widget like a button (useful at the bottom of forms).

Dialogs

QFileDialog

- **QFileDialog** opens a native file browser dialog to let users select files.

- Useful for tasks like loading an image, config file, or data file into your tool.

```

1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QFileDialog,
  QPushButton
3
4 class MainWindow(QMainWindow):
5     def __init__(self):
6         super().__init__()
7         self.setWindowTitle("QFileDialog Example")
8
9         self.button = QPushButton("Open File")
10        self.button.clicked.connect(self.open_file_dialog)
11        self.setCentralWidget(self.button)
12
13    def open_file_dialog(self):
14        file_name, _ = QFileDialog.getOpenFileName(
15            self,
16            "Open File",
17            "",
18            "All Files (*);;Text Files (*.txt)"
19        )
20        if file_name:
21            print(file_name)
22
23 if __name__ == "__main__":
24     app = QApplication(sys.argv)
25     window = MainWindow()
26     window.show()
27     app.exec_()
28

```

Explanation

• `QFileDialog.getOpenFileName(...)`

- Opens the native file dialog
- Parameters:
 - `"Open File"` → Dialog window title
 - `""` → Opens in the default directory
 - `"All Files (*);;Text Files (*.txt)"` → File type filters shown in the dropdown

• `file_name, _ = ...`

- Returns a tuple:
 - `file_name` : full path of the selected file
 - `_` : selected filter (not used, so stored as `_`)

• `if file_name:`

- Checks whether the user selected a file or canceled the dialog

QColorDialog

- `QColorDialog` opens a standard color picker dialog that allows the user to select a color.
- Useful for UI theming, paint tools, lighting controls, or setting custom overlay colors.

```

1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QColorDialog,
  QPushButton
3

```

```

4 class MainWindow(QMainWindow):
5     def __init__(self):
6         super().__init__()
7         self.setWindowTitle("QColorDialog Example")
8
9         self.button = QPushButton("Choose Color")
10        self.button.clicked.connect(self.open_color_dialog)
11        self.setCentralWidget(self.button)
12
13    def open_color_dialog(self):
14        color = QColorDialog.getColor()
15        if color.isValid():
16            print(color.name())
17
18 if __name__ == "__main__":
19     app = QApplication(sys.argv)
20     window = MainWindow()
21     window.show()
22     app.exec_()

```

Explanation

- **QColorDialog.getColor()**
 - Opens the color picker dialog
 - Returns a **QColor** object
 - If the user cancels, it returns an invalid color
- **color.isValid()**
 - Checks whether the user selected a valid color
 - Useful to prevent using empty or canceled inputs
- **color.name()**
 - Returns the selected color in hex format (e.g. **#ff9900**)
 - Helpful for storing or displaying the selected color

QFontDialog

- **QFontDialog** opens a font selection dialog, allowing the user to choose a font family, size, and style.
- Useful in tools where users configure overlays, slates, or UI themes.

```

1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QFontDialog,
  QPushButton
3
4 class MainWindow(QMainWindow):
5     def __init__(self):
6         super().__init__()
7         self.setWindowTitle("QFontDialog Example")
8
9         self.button = QPushButton("Choose Font")
10        self.button.clicked.connect(self.open_font_dialog)
11        self.setCentralWidget(self.button)
12
13    def open_font_dialog(self):
14        font, ok = QFontDialog.getFont()
15        if ok:
16            print(font.toString())
17
18 if __name__ == "__main__":
19     app = QApplication(sys.argv)
20     window = MainWindow()

```

```
21     window.show()
22     app.exec_()
23
```

Explanation

- **QFontDialog.getFont()**
 - Opens the system font selection dialog
 - Returns:
 - **font** : the selected **QFont** object
 - **ok** : a boolean — **True** if the user clicked OK
- **font.toString()**
 - Converts the selected font into a readable string
 - Useful for saving or displaying the chosen font settings
- **ok**
 - Ensures that the font is only used if the user confirmed the dialog

Widgets L2

QMainWindow

- **QMainWindow** is a common starting point for building full-featured desktop tools.
- It supports built-in areas like menus, toolbars, status bars, and a central widget.

```
1  import sys
2  from PySide2.QtCore import QSize, Qt
3  from PySide2.QtWidgets import QApplication, QMainWindow, QLabel,
   QMainWindow, QMenuBar, QToolBar, QAction
4
5  class MainWindow(QMainWindow):
6      def __init__(self):
7          super().__init__()
8          self.setWindowTitle("MainWindow Example")
9
10         label = QLabel("Hello, World!")
11         self.setCentralWidget(label)
12
13         menu_bar = QMenuBar(self)
14         self.setMenuBar(menu_bar)
15
16         file_menu = menu_bar.addMenu("File")
17         exit_action = QAction("Exit", self)
18         exit_action.triggered.connect(self.close)
19         file_menu.addAction(exit_action)
20
21         tool_bar = QToolBar(self)
22         self.addToolBar(tool_bar)
23         tool_bar.addAction(exit_action)
24
25  if __name__ == "__main__":
26      app = QApplication(sys.argv)
27      window = MainWindow()
28      window.show()
29      app.exec_()
```

QPixmap

- `QPixmap` is used to handle and display images in Qt.
- It's optimized for **displaying images on screen**, especially in widgets like `QLabel`.

```
1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QLabel
3 from PySide2.QtGui import QPixmap
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QPixmap Example")
9
10        pixmap = QPixmap("sample_image.jpg") # Replace with an actual
        image path
11
12        label = QLabel()
13        label.setPixmap(pixmap)
14        label.setScaledContents(True) # Optional: scale the image to
        fit label size
15
16        self.setCentralWidget(label)
17
18 if __name__ == "__main__":
19     app = QApplication(sys.argv)
20     window = MainWindow()
21     window.resize(400, 300)
22     window.show()
23     app.exec_()
24
```

Explanation

- `QPixmap("sample_image.jpg")`
 - Loads an image from the given file path
 - Supported formats include PNG, JPG, BMP, etc.
 - This is a **non-editable** image class (use `QImage` if you need pixel manipulation)
- `QLabel.setPixmap(pixmap)`
 - Assigns the loaded pixmap to a label so it can be displayed in the UI
- `label.setScaledContents(True)`
 - Optional setting that allows the image to resize with the label
 - Without this, the image retains its original size

QSpinBox

- `QSpinBox` provides a small box with up/down arrows for selecting **integer values**.
- Commonly used in forms, settings panels, or any place you need numeric input with a fixed range.

```
1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QSpinBox
3
4 class MainWindow(QMainWindow):
5     def __init__(self):
6         super().__init__()
7         self.setWindowTitle("QSpinBox Example")
```

```

8
9     self.spin_box = QSpinBox()
10    self.spin_box.setRange(0, 100)
11    self.spin_box.setValue(10)
12
13    self.setCentralWidget(self.spin_box)
14
15 if __name__ == "__main__":
16     app = QApplication(sys.argv)
17     window = MainWindow()
18     window.show()
19     app.exec_()
20

```

Explanation

- **QSpinBox()**

- Widget for integer input
- Displays a number field with up/down arrow controls

- **setRange(min, max)**

- Defines the minimum and maximum allowed values
- In this case, user can select any number from **0 to 100**

- **setValue(10)**

- Sets the initial value shown in the spin box

- **setCentralWidget(self.spin_box)**

- Displays the spin box in the center of the main window

QScrollArea

- **QScrollArea** provides scrollbars when its child widget is **larger than the available visible area**.
- Useful for custom UIs or tools where you have more content than space — like large forms, long lists, or dynamic settings panels.

```

1 import sys
2 from PySide2.QtWidgets import (
3     QApplication, QMainWindow, QScrollArea, QWidget,
4     QVBoxLayout, QLabel, QPushButton
5 )
6
7 class MainWindow(QMainWindow):
8     def __init__(self):
9         super().__init__()
10        self.setWindowTitle("QScrollArea Example")
11
12        # Create a widget with a vertical layout (this will go inside
the scroll area)
13        content_widget = QWidget()
14        layout = QVBoxLayout(content_widget)
15
16        # Add many widgets to simulate large content
17        for i in range(30):
18            layout.addWidget(QLabel(f"Label {i+1}"))
19            layout.addWidget(QPushButton(f"Button {i+1}"))
20
21        # Scroll Area setup
22        scroll_area = QScrollArea()
23        scroll_area.setWidgetResizable(True) # Let content resize
with scroll area
24        scroll_area.setWidget(content_widget)

```

```

25
26         self.setCentralWidget(scroll_area)
27
28     if __name__ == "__main__":
29         app = QApplication(sys.argv)
30         window = MainWindow()
31         window.resize(300, 400)
32         window.show()
33         app.exec_()

```

Explanation

- **QScrollArea()**
 - A container that adds scrollbars when needed
 - Use it when your content might be taller or wider than the window
- **content_widget = QWidget()**
 - The main widget that will hold all scrollable content
 - Must be set inside the scroll area using `setWidget()`
- **QVBoxLayout(content_widget)**
 - Adds a vertical layout to the content widget
 - Allows stacking labels and buttons vertically
- **setWidgetResizable(True)**
 - Ensures that the scroll area adjusts its size based on available space
 - Recommended when the scroll area is inside resizable windows
- **scroll_area.setWidget(content_widget)**
 - Loads the scrollable content into the scroll area
- **setCentralWidget(scroll_area)**
 - Places the scroll area into the main window layout

QDoubleSpinBox

- **QDoubleSpinBox** is just like **QSpinBox**, but it supports **floating-point numbers (decimals)**.
- Useful for setting values like scale, opacity, frame rate, or precision settings in VFX tools.

```

1  import sys
2  from PySide2.QtWidgets import QApplication, QMainWindow,
   QDoubleSpinBox
3
4  class MainWindow(QMainWindow):
5      def __init__(self):
6          super().__init__()
7          self.setWindowTitle("QDoubleSpinBox Example")
8
9          self.double_spin_box = QDoubleSpinBox()
10         self.double_spin_box.setRange(0.0, 100.0)
11         self.double_spin_box.setValue(10.5)
12
13         self.setCentralWidget(self.double_spin_box)
14
15     if __name__ == "__main__":
16         app = QApplication(sys.argv)
17         window = MainWindow()
18         window.show()
19         app.exec_()
20

```

Explanation

- **QDoubleSpinBox()**
 - Widget for inputting decimal (floating-point) values
 - Includes up/down arrows for step adjustment
- **setRange(0.0, 100.0)**
 - Sets the minimum and maximum float values the user can select
- **setValue(10.5)**
 - Sets the initial value displayed when the widget loads
- **setCentralWidget(...)**
 - Displays the spin box as the main widget in the window

QSplitter

- **QSplitter** is a container that lets the user **resize two or more widgets** by dragging a handle between them.
- It's useful for side-by-side views like editors, file explorers, or outline panels.

```

1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QSplitter,
  QTextEdit
3 from PySide2.QtCore import Qt
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QSplitter Example")
9
10        self.splitter = QSplitter(Qt.Horizontal)
11
12        self.editor1 = QTextEdit()
13        self.editor2 = QTextEdit()
14
15        self.splitter.addWidget(self.editor1)
16        self.splitter.addWidget(self.editor2)
17
18        self.setCentralWidget(self.splitter)
19
20 if __name__ == "__main__":
21     app = QApplication(sys.argv)
22     window = MainWindow()
23     window.show()
24     app.exec_()

```

Explanation

- **QSplitter(Qt.Horizontal)**
 - Creates a splitter that arranges child widgets horizontally (side-by-side)
 - You can also use **Qt.Vertical** to stack widgets vertically
- **addWidget(...)**
 - Adds widgets (like **QTextEdit**) to the splitter
 - Each added widget gets its own resizable panel
- **QTextEdit()**
 - Used here as placeholder content in each panel

- **setCentralWidget(self.splitter)**
 - Makes the splitter the main layout container in the window

QDockWidget

- **QDockWidget** is a panel that can **dock, undock, or float** around the main window.
- Commonly used in VFX tools for side panels like settings, logs, property editors, or metadata views.

```

1 import sys
2 from PySide2.QtWidgets import QApplication, QMainWindow, QDockWidget,
  QTextEdit
3 from PySide2.QtCore import Qt
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QDockWidget Example")
9
10        self.editor = QTextEdit()
11        self.setCentralWidget(self.editor)
12
13        self.dock = QDockWidget("Dockable Panel", self)
14        self.dock.setWidget(QTextEdit())
15        self.addDockWidget(Qt.RightDockWidgetArea, self.dock)
16
17 if __name__ == "__main__":
18     app = QApplication(sys.argv)
19     window = MainWindow()
20     window.show()
21     app.exec_()

```

Explanation

- **QDockWidget("Dockable Panel", self)**
 - Creates a dockable window with the title "Dockable Panel"
 - Attached to the main window (**self**)
- **setWidget(QTextEdit())**
 - Sets the content of the dock panel
 - You can use any widget here, such as lists, controls, or forms
- **addDockWidget(Qt.RightDockWidgetArea, self.dock)**
 - Places the dock on the right side of the main window
 - You can change **RightDockWidgetArea** to **Left** , **Top** , or **Bottom**
- **setCentralWidget(self.editor)**
 - The main text editor in the center of the window

View Widgets

QListView

- **QListView** displays a **list of items** using a data model.

- Useful when you want to present a dynamic or selectable list of text items in your tool (like task lists, asset names, etc.).

```

1 import sys
2 from PySide2.QtCore import QStringListModel
3 from PySide2.QtWidgets import QApplication, QMainWindow, QListView
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QListView Example")
9
10        self.list_view = QListView()
11        self.model = QStringListModel(["Item 1", "Item 2", "Item 3"])
12        self.list_view.setModel(self.model)
13        self.setCentralWidget(self.list_view)
14
15 if __name__ == "__main__":
16     app = QApplication(sys.argv)
17     window = MainWindow()
18     window.show()
19     app.exec_()

```

Explanation

• QStringListModel(...)

- A simple data model that holds a list of strings
- Works as the data source for the `QListView`

• list_view.setModel(model)

- Connects the view to the model
- The view will automatically display all items in the model

• QListView

- A widget that displays items vertically
- Can be extended to support item selection, editing, or custom views later

QTreeView

- `QTreeView` displays **hierarchical (tree-like) data**, where items can have parent-child relationships.
- Useful for representing folder structures, asset hierarchies, or grouped data in tools.

```

1 import sys
2 from PySide2.QtGui import QStandardItemModel, QStandardItem
3 from PySide2.QtWidgets import QApplication, QMainWindow, QTreeView
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QTreeView Example")
9
10        self.tree_view = QTreeView()
11        self.model = QStandardItemModel()
12        self.model.setHorizontalHeaderLabels(["Header"])
13        self.tree_view.setModel(self.model)
14
15        root_node = self.model.invisibleRootItem()
16        item0 = QStandardItem("Item 0")
17        item1 = QStandardItem("Item 1")
18        item0.appendRow(QStandardItem("Subitem 0"))
19        item1.appendRow(QStandardItem("Subitem 1"))

```

```

20
21     root_node.appendRow(item0)
22     root_node.appendRow(item1)
23
24     self.setCentralWidget(self.tree_view)
25
26 if __name__ == "__main__":
27     app = QApplication(sys.argv)
28     window = MainWindow()
29     window.show()
30     app.exec_()
31

```

Explanation

- **QTreeView**
 - A widget used to display hierarchical data in a tree format
 - Supports expanding/collapsing parent nodes
- **QStandardItemModel()**
 - Provides a default data model for **QTreeView**
 - Can contain items in a tree (parent-child) structure
- **model.setHorizontalHeaderLabels(["Header"])**
 - Sets the header label for the column (optional but recommended)
- **model.invisibleRootItem()**
 - Returns the root node (not visible in the UI)
 - We use this to append top-level items
- **QStandardItem("Item 0") / appendRow(...)**
 - Creates items with text content
 - **appendRow()** is used to:
 - Add sub-items to a parent (for nesting)
 - Add top-level items to the root node

QTableView

- **QTableView** displays data in a **spreadsheet-like table** using rows and columns.
- Useful for showing asset lists, shot data, render stats, or publish results in tools.

```

1 import sys
2 from PySide2.QtGui import QStandardItemModel, QStandardItem
3 from PySide2.QtWidgets import QApplication, QMainWindow, QTableView
4
5 class MainWindow(QMainWindow):
6     def __init__(self):
7         super().__init__()
8         self.setWindowTitle("QTableView Example")
9
10        self.table_view = QTableView()
11        self.model = QStandardItemModel(4, 3)
12        self.model.setHorizontalHeaderLabels(["Column 1", "Column 2",
"Column 3"])
13        self.table_view.setModel(self.model)
14
15        for row in range(4):
16            for column in range(3):
17                item = QStandardItem(f"Item {row},{column}")

```

```
18         self.model.setItem(row, column, item)
19
20         self.setCentralWidget(self.table_view)
21
22     if __name__ == "__main__":
23         app = QApplication(sys.argv)
24         window = MainWindow()
25         window.show()
26         app.exec_()
```

Explanation

- **QTableView**

- A widget to display tabular data (rows and columns)
- Can show data from any compatible model, such as `QStandardItemModel`

- **QStandardItemModel(4, 3)**

- Creates a model with 4 rows and 3 columns
- Acts as the data source for the table

- **model.setHorizontalHeaderLabels([...])**

- Sets column names shown at the top of the table

- **for row in range(4), for column in range(3)**

- Loops through the grid to fill each cell with an item
- Each item is created using `QStandardItem("...")`

- **model.setItem(row, column, item)**

- Places each item in the correct row and column of the model

- **table_view.setModel(self.model)**

- Connects the model to the table view so the data is shown